

Why Ray

Machine
learning is
pervasive

Distributed
computing is a
necessity

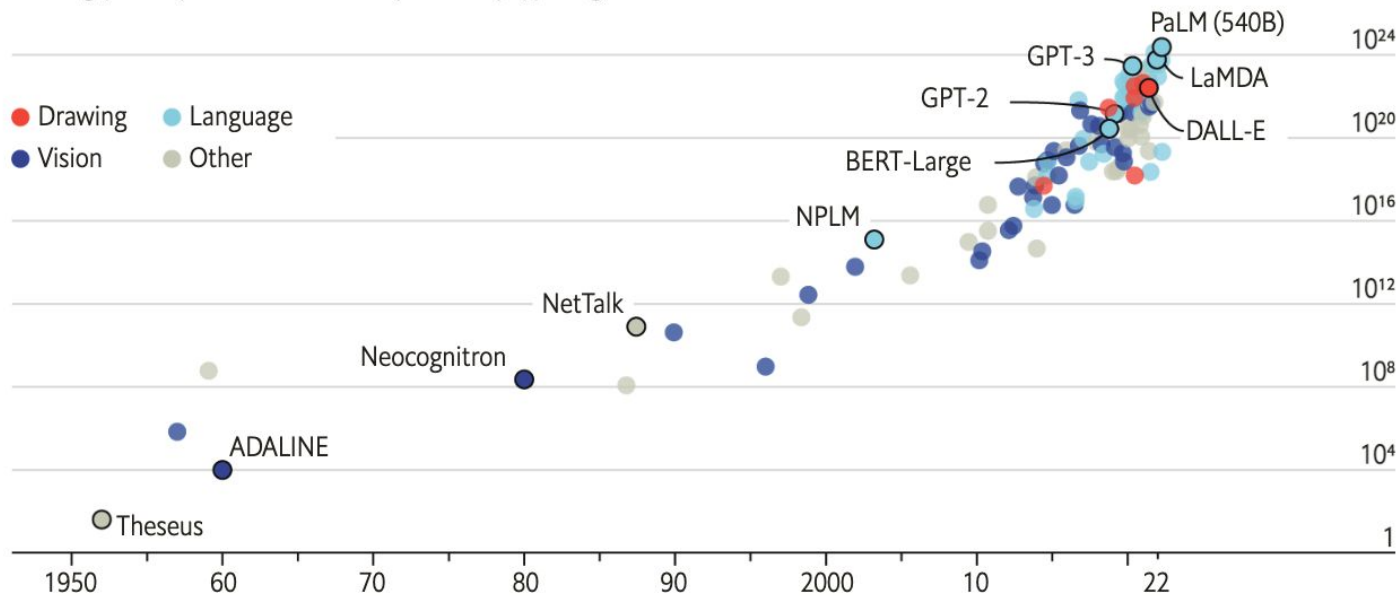
Python is the
default
language for
DS/ML

Blessings of scale ...

The blessings of scale

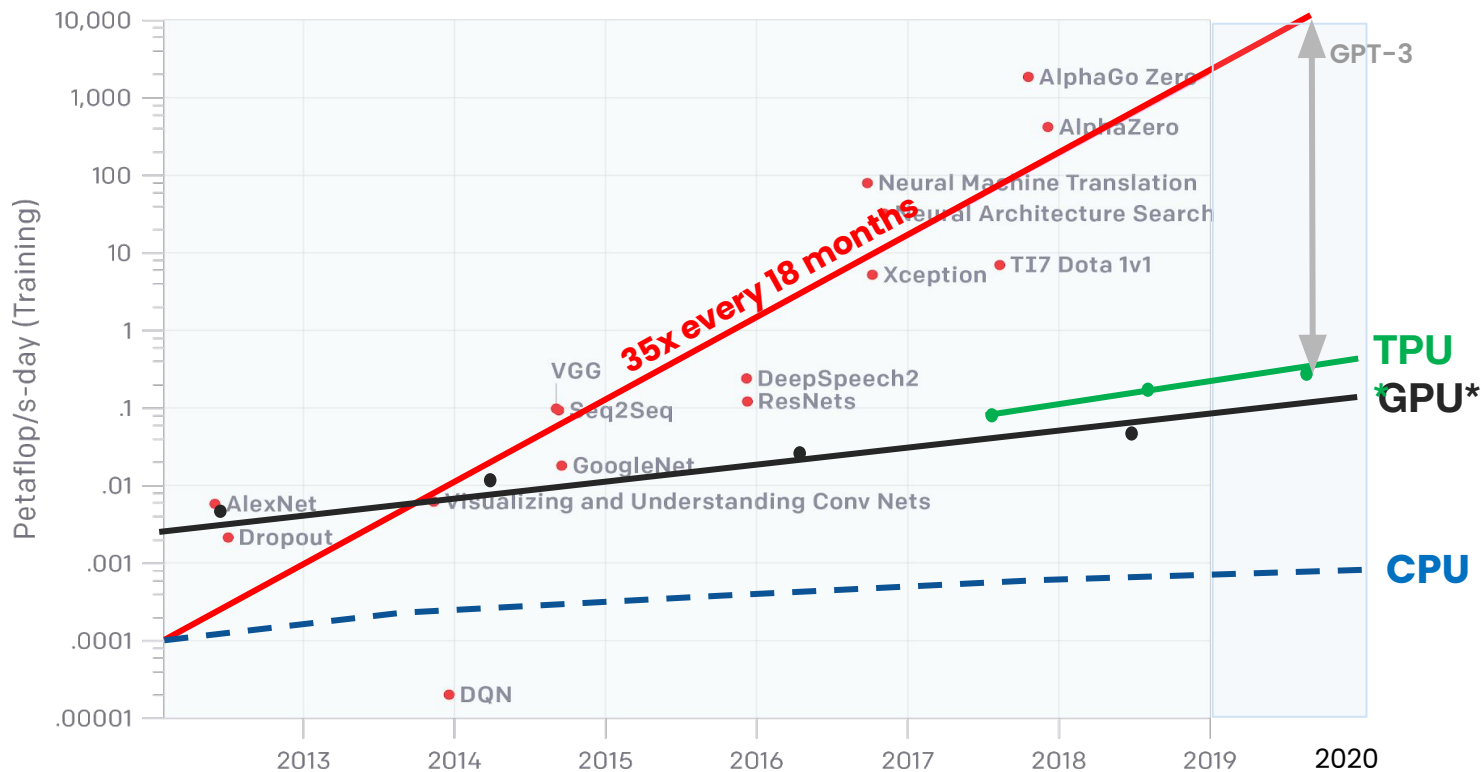
AI training runs, estimated computing resources used

Floating-point operations, selected systems, by type, log scale

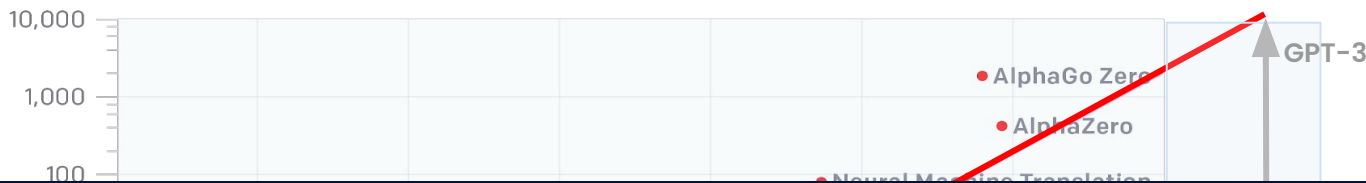


Sources: "Compute trends across three eras of machine learning", by J. Sevilla et al., arXiv, 2022; Our World in Data

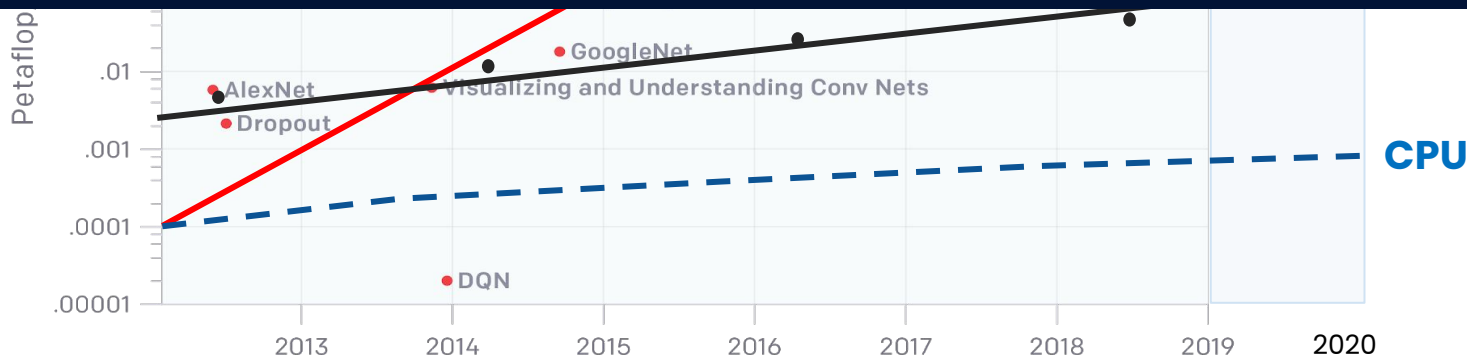
Compute - supply demand problem



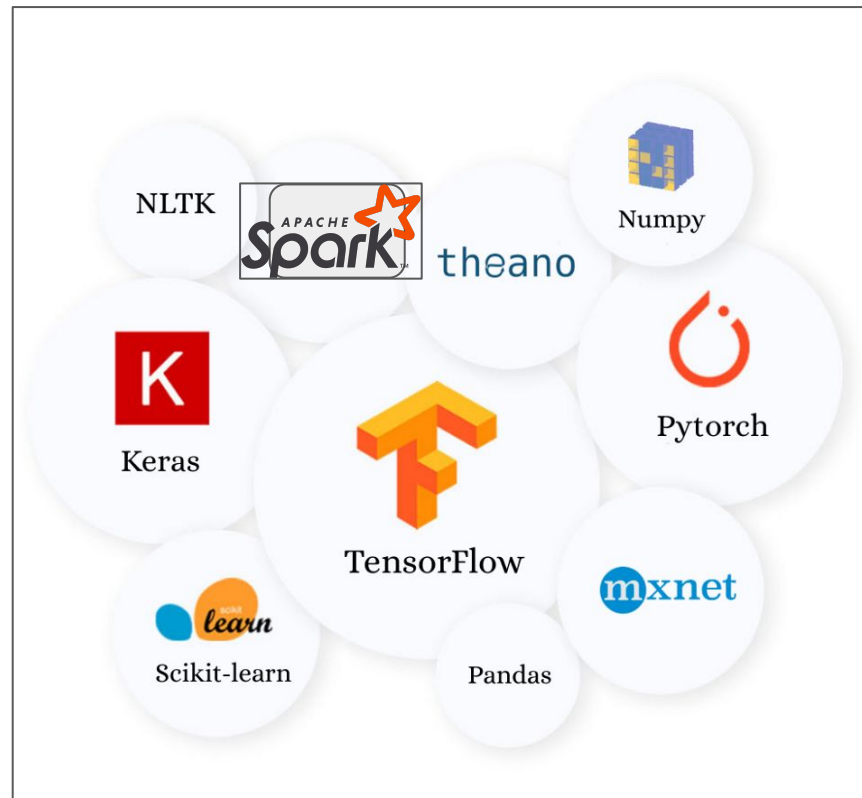
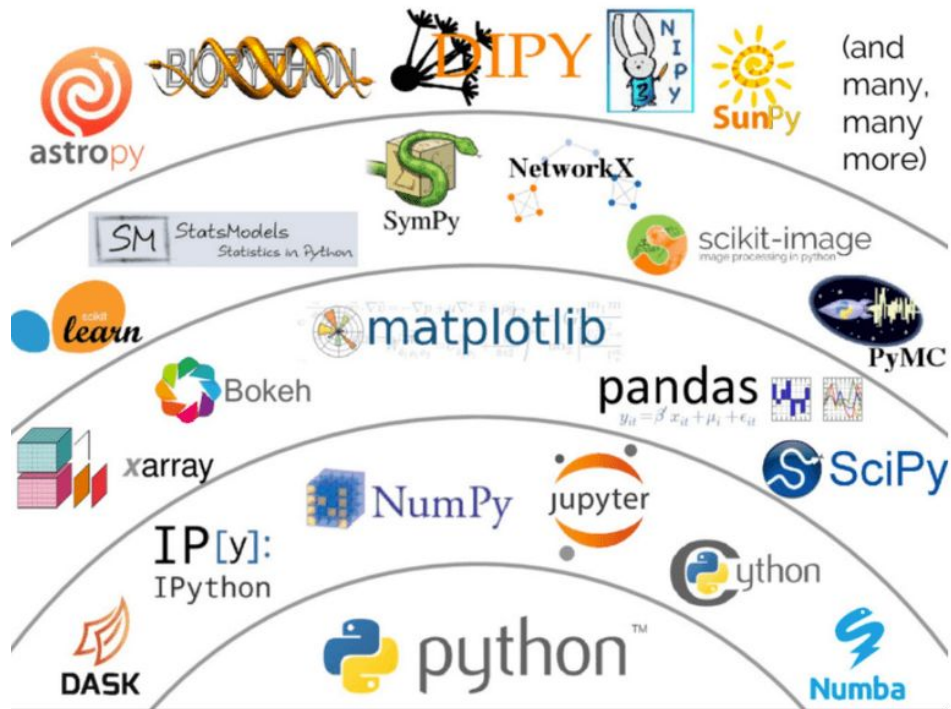
Specialized hardware is not enough



No way out but to distribute!



Python data science ecosystem

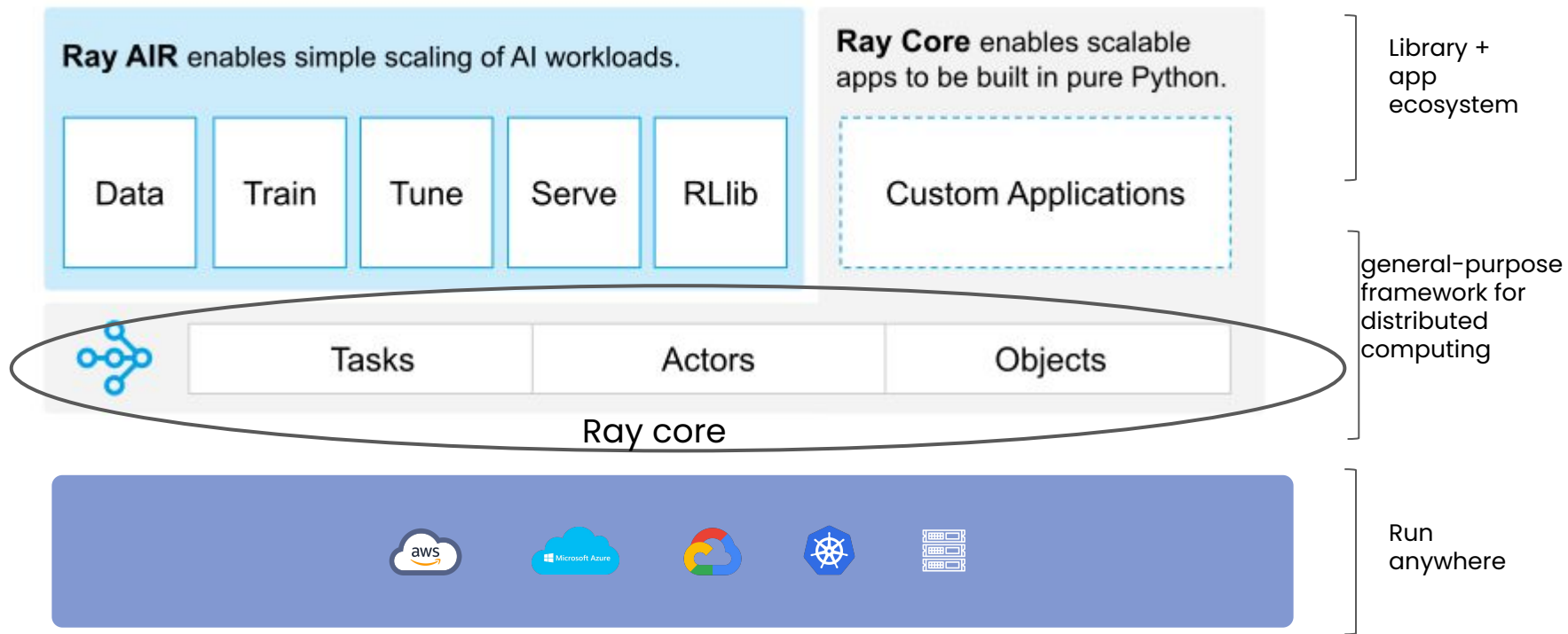


What is Ray

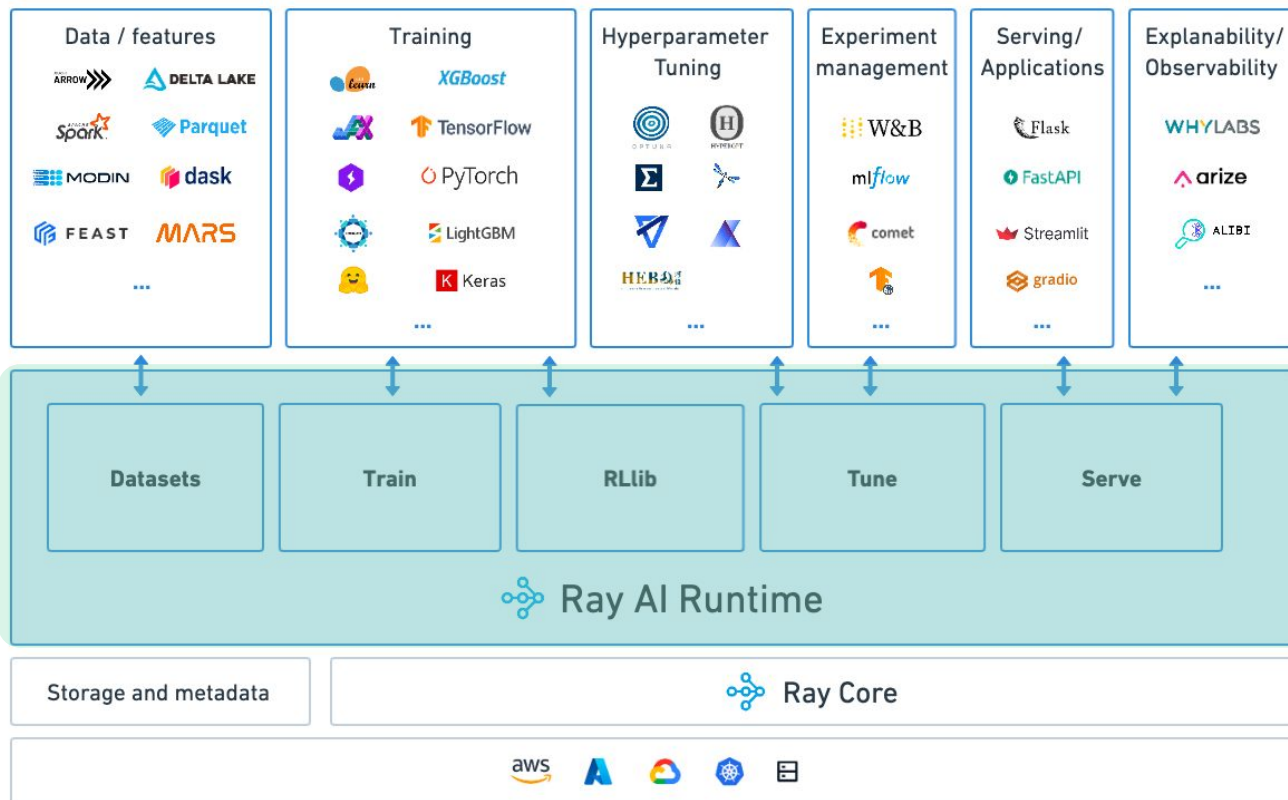
- A ***simple/general-purpose*** library for distributed computing
- An ecosystem of Python libraries (for scaling ML and more)
- Runs on laptop, public cloud, K8s, on-premise

A layered cake of functionality and capabilities for scaling ML workloads

A Layered Cake and Ecosystem

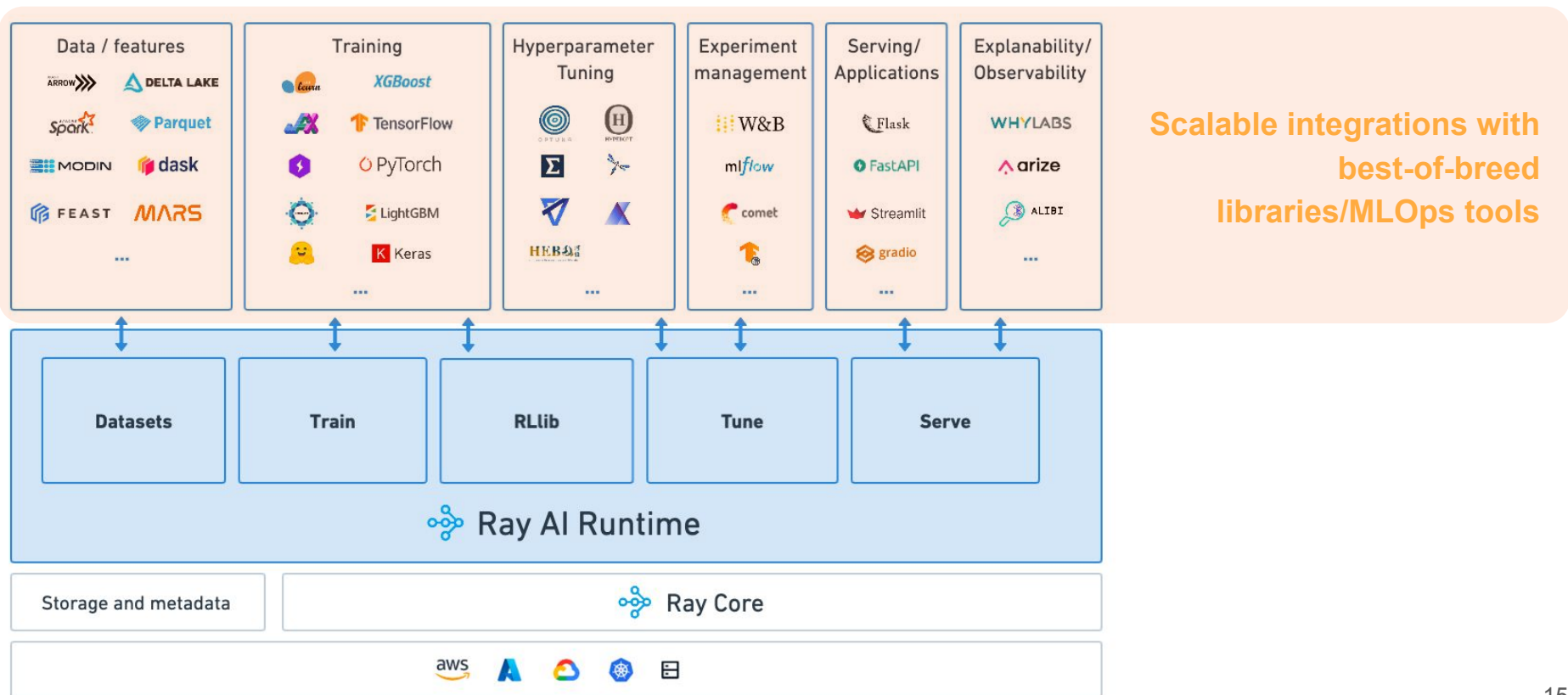


Ray AI Runtime (AIR) is a scalable runtime for end-to-end ML applications

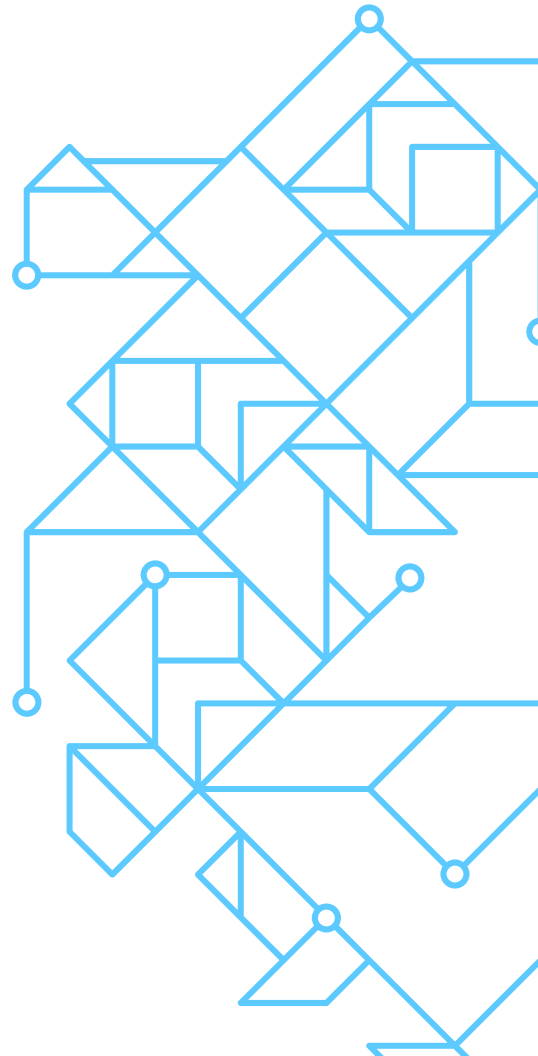


High-level libraries that make scaling easy for both data scientists and ML engineers.

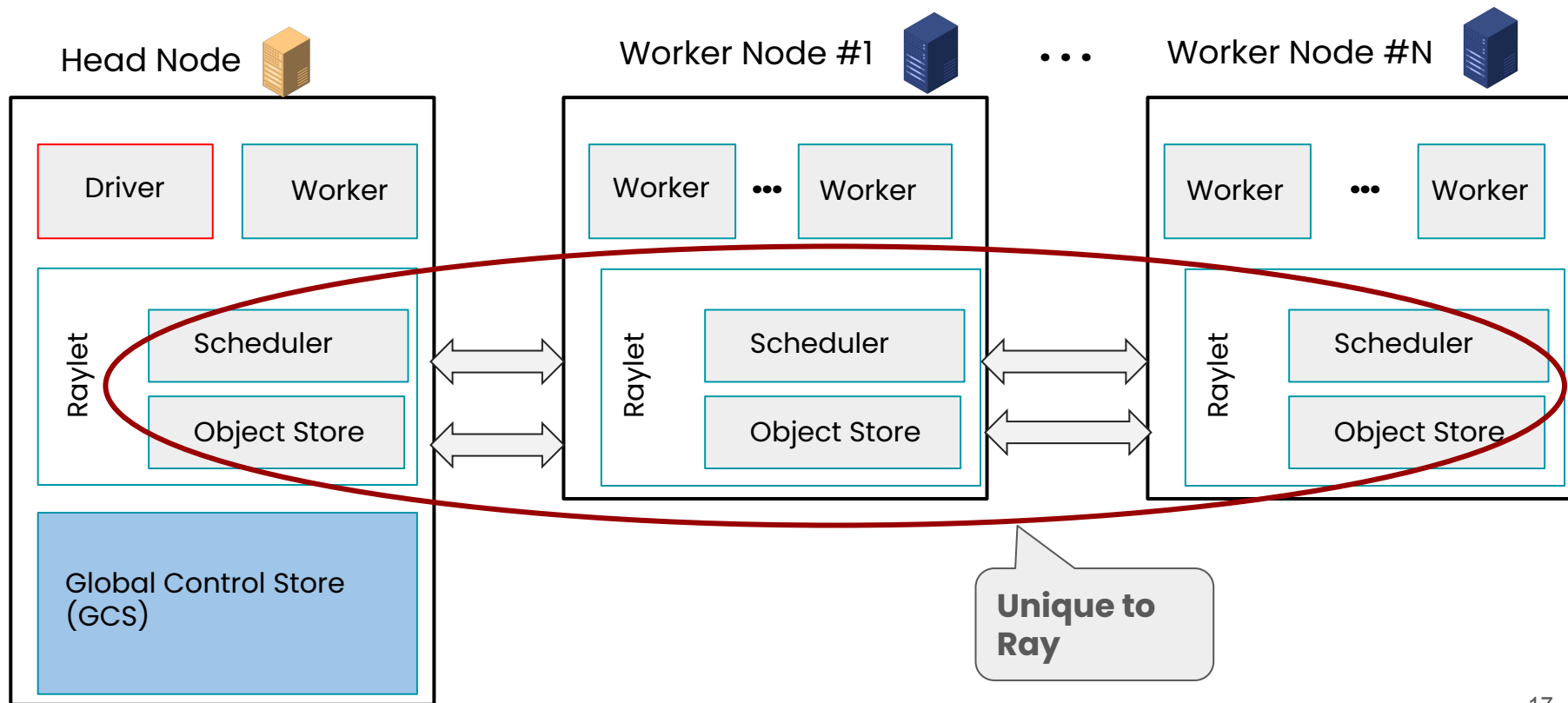
Ray AI Runtime (AIR) is a scalable toolkit for end-to-end ML applications



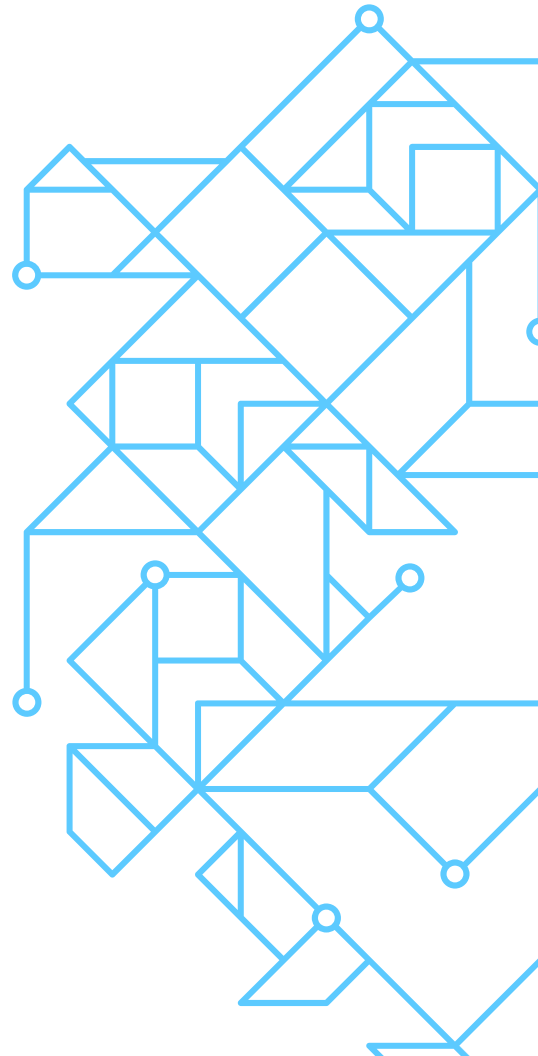
Ray Architecture & Components



An anatomy of a Ray cluster



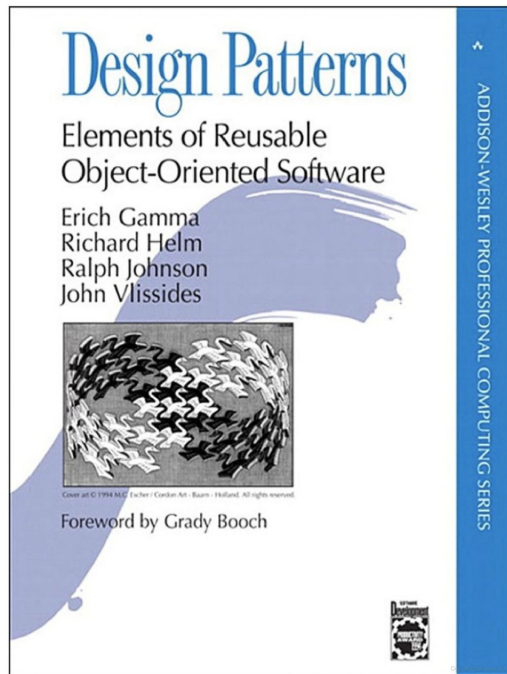
Ray distributed design & scaling patterns & APIs



Ray Basic Design Patterns

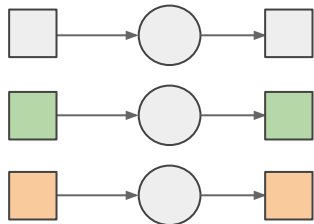
- Ray Parallel Tasks
 - Functions as stateless units of execution
 - Functions distributed across the cluster as tasks
- Ray Objects as Futures
 - Distributed (immutable objects) store in the cluster
 - Fetched when materialized
 - Enable massive asynchronous parallelism
- Ray Actors
 - Stateful service on a cluster
 - Enable Message passing

1. [Patterns for Parallel Programming](#)
2. [Ray Design Patterns](#)
3. [Ray Distributed Library Integration Patterns](#)



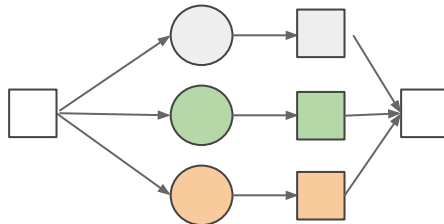
Scaling Design Patterns

Batch Training / Inference



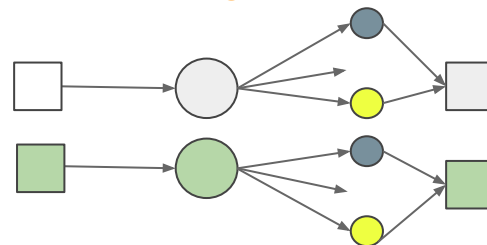
Different data / Same function

AutoML



Same data / Different function

Batch Tuning



Different data / Same function /

○ Compute

□ Data

Python → Ray APIs



```
def f(x):  
    # do something with  
    x:  
    y = ...  
    return y
```

Task



```
@ray.remote  
def f(x):  
    # do something with  
    x:  
    y = ...  
    return y  
f.remote()
```

Distributed



```
class Cls():  
    def  
    __init__(self, x):  
    def f(self, a):  
        ...  
    def g(self, a):  
        ...
```

Actor



```
@ray.remote  
class Cls():  
    def __init__(self,  
    x):  
    def f(self, a):  
        ...  
    def g(self, a):  
        ...  
cls = Cls.remote()  
cls.f.remote(a)
```

Distributed



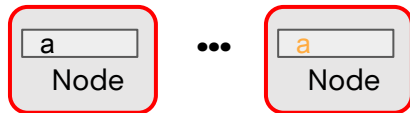
```
import numpy as np  
a = np.arange(1, 10e6)  
b = a * 2
```

Distributed
immutable
object



```
import numpy as np  
a = np.arange(1, 10e6)  
obj_a = ray.put(a)  
b = ray.get(obj_a) * 2
```

Distributed

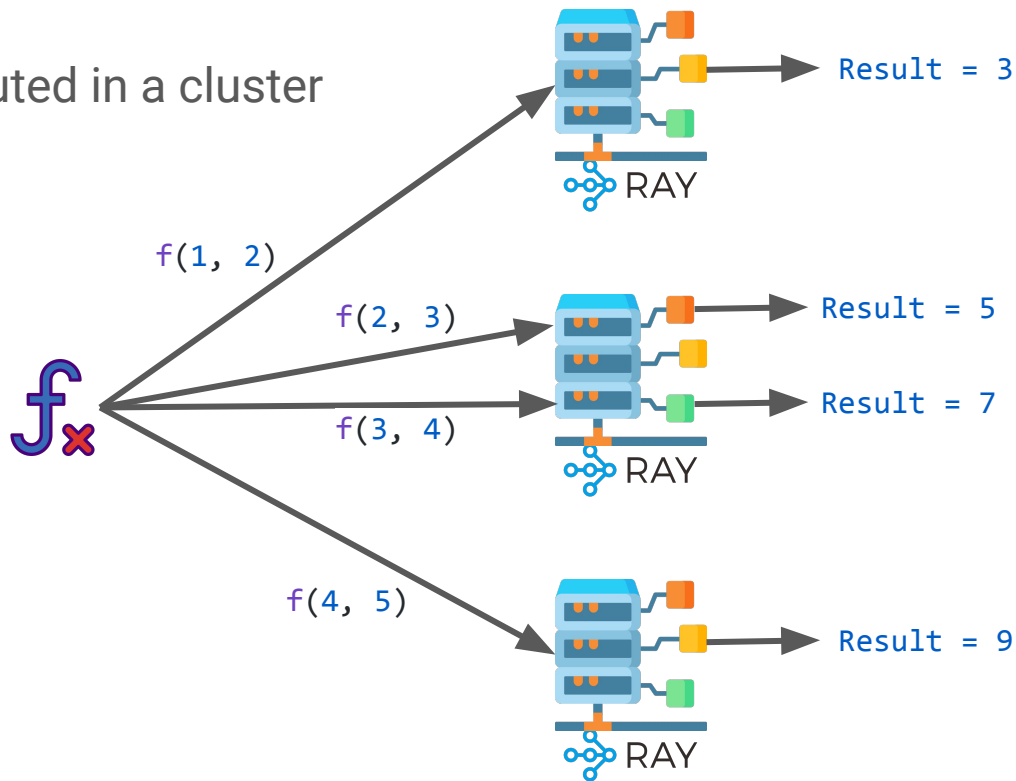


Ray Task

A function f_x remotely executed in a cluster

```
@ray.remote(num_cpus=2)
def f(a, b):
    return a + b

f.remote(1, 2) # returns 3
```



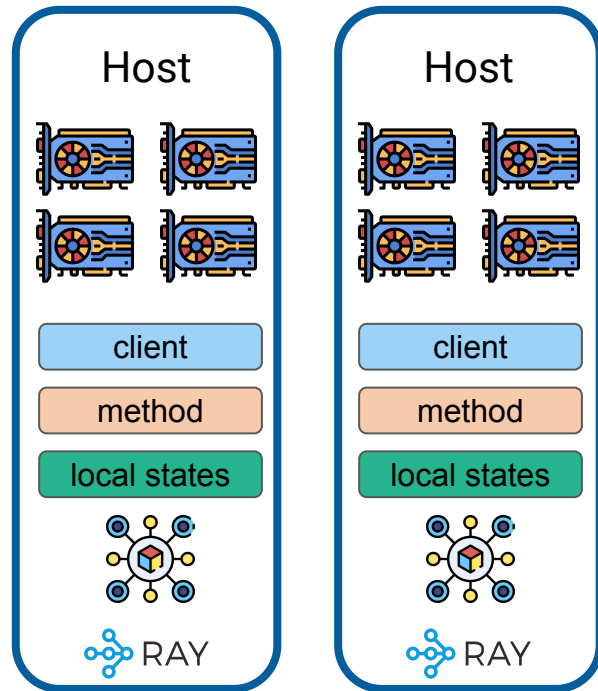
Ray Actor

A  class remotely executed in a cluster

```
@ray.remote(num_gpus=4)
class HostActor:
    def __init__(self):
        self.num_devices = os.environ["CUDA_VISIBLE_DEVICES"]

    def f(self, output):
        return f"{output} {self.num_devices}"

actor = HostActor.remote() # Create an actor
actor.f.remote("hi") # returns "hi 0,1,2,3"
```



Function → Task

```
@ray.remote
def read_array(file):
    # read ndarray "a"
    # from "file"
    return a
```

```
@ray.remote
def add(a, b):
    return np.add(a, b)
```

```
id1 = read_array.remote(file1)
id2 = read_array.remote(file2)
id = add.remote(id1, id2)
sum = ray.get(id)
```

Class → Actor

```
@ray.remote(num_gpus=1)
class Counter(object):
    def __init__(self):
        self.value = 0
    def inc(self):
        self.value += 1
        return self.value
```

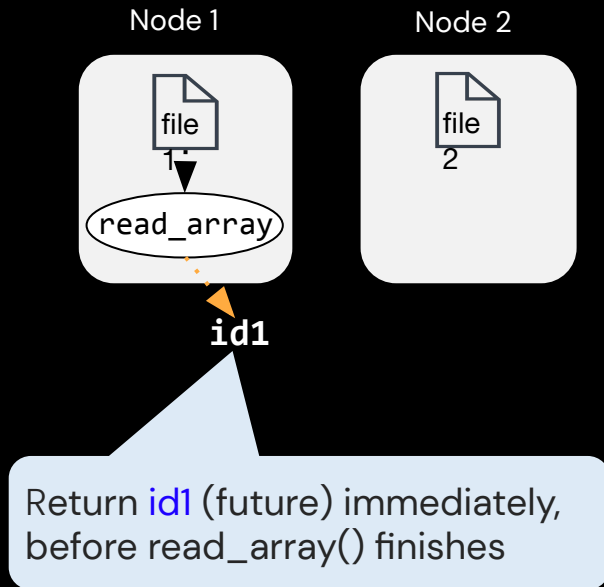
```
c = Counter.remote()
id4 = c.inc.remote()
id5 = c.inc.remote()
```

Task API

```
@ray.remote
def read_array(file):
    # read ndarray "a"
    # from "file"
    return a

@ray.remote
def add(a, b):
    return np.add(a, b)

id1 = read_array.remote(file1)
id2 = read_array.remote(file2)
id = add.remote(id1, id2)
sum = ray.get(id)
```

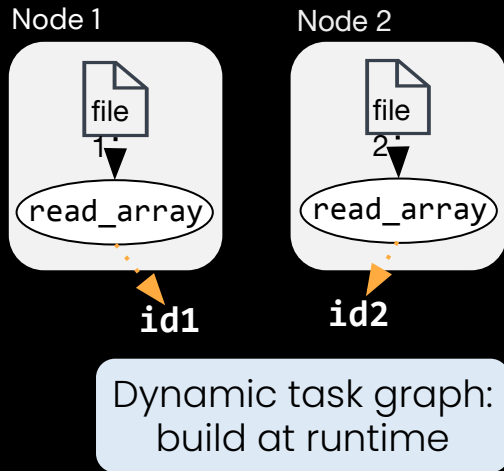


Task API

```
@ray.remote
def read_array(file):
    # read ndarray "a"
    # from "file"
    return a

@ray.remote
def add(a, b):
    return np.add(a, b)

id1 = read_array.remote(file1)
id2 = read_array.remote(file2)
id = add.remote(id1, id2)
sum = ray.get(id)
```



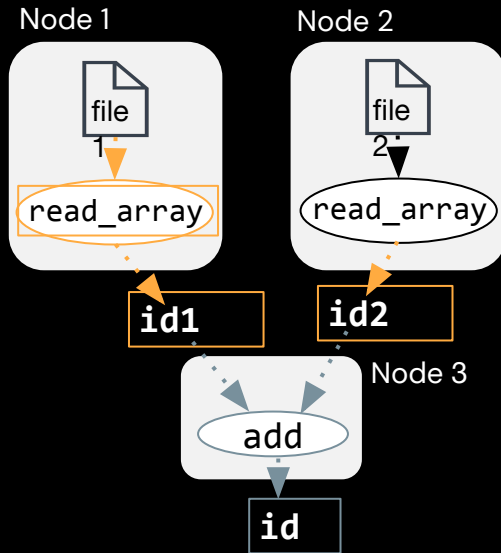
Task API

```
@ray.remote
def read_array(file):
    # read ndarray "a"
    # from "file"
    return a
```

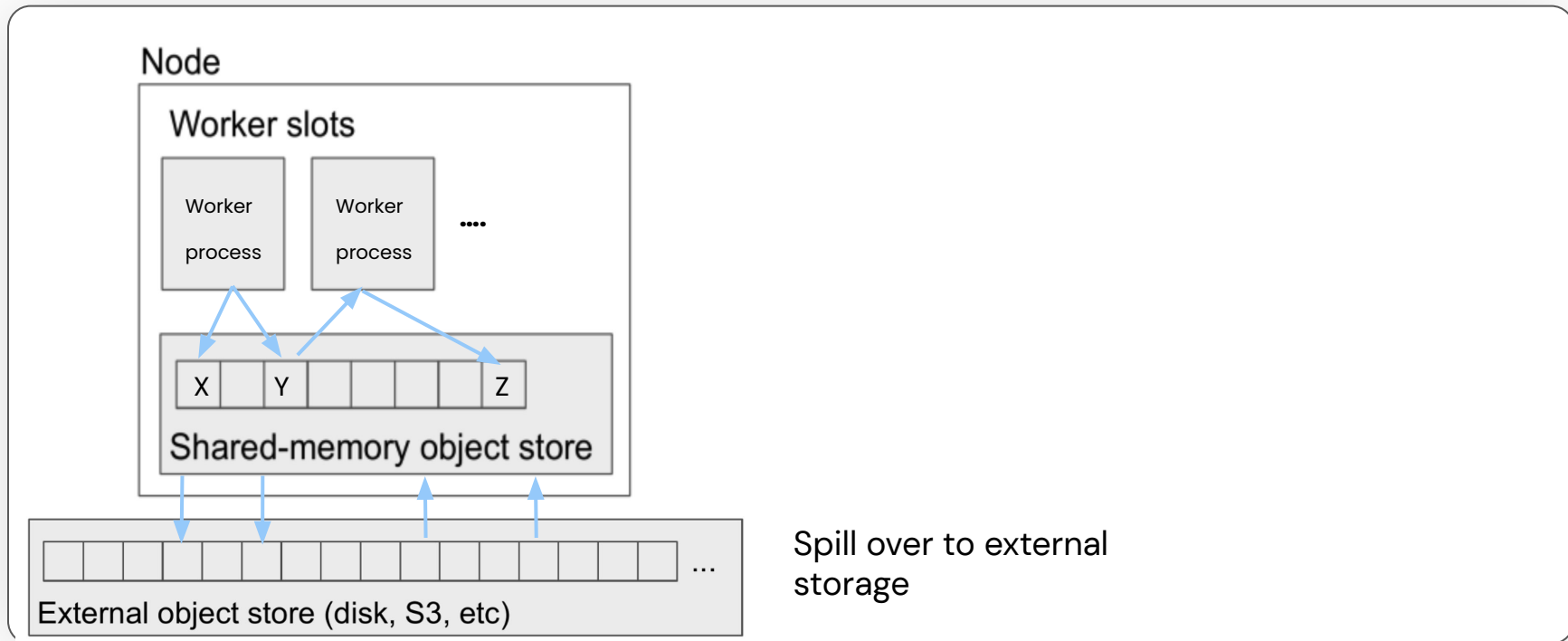
```
@ray.remote
def add(a, b):
    return np.add(a, b)
```

```
id1 = read_array.remote(file1)
id2 = read_array.remote(file2)
id = add.remote(id1, id2)
sum = ray.get(id)
```

`ray.get()` block until
result available



Distributed Immutable object store



Distributed object store

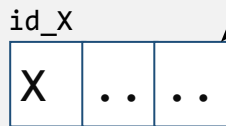
Node 1

```
@ray.remote  
def f():  
    ...  
    return X
```

```
@ray.remote  
def g(a):  
    ...  
    return Y
```

```
id_X = f.remote()  
id_Y = g.remote(id_X)
```

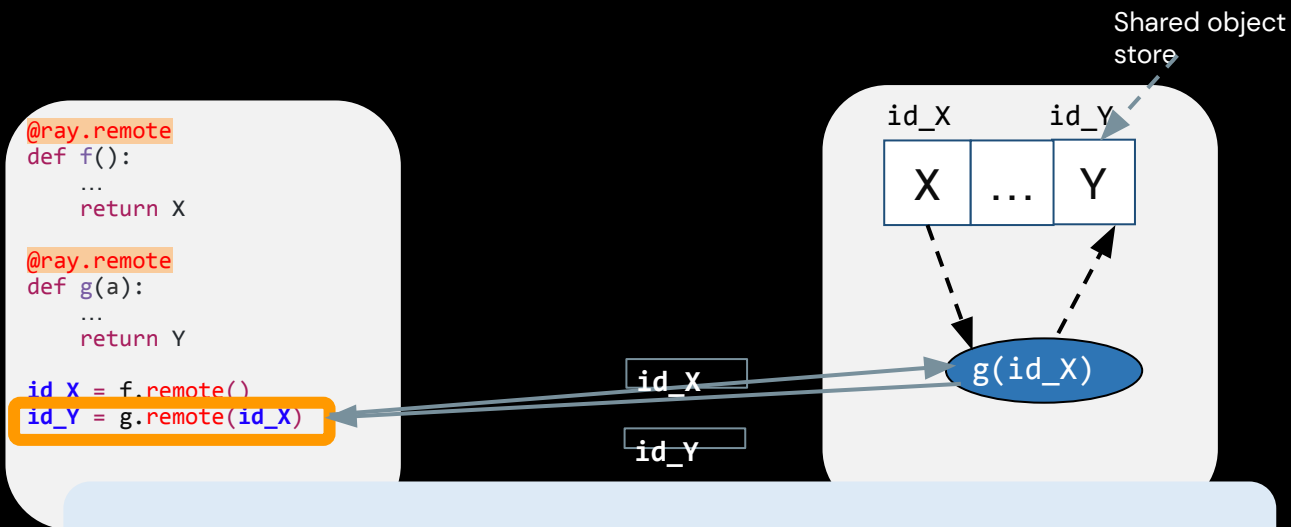
Node 2



Shared object
store

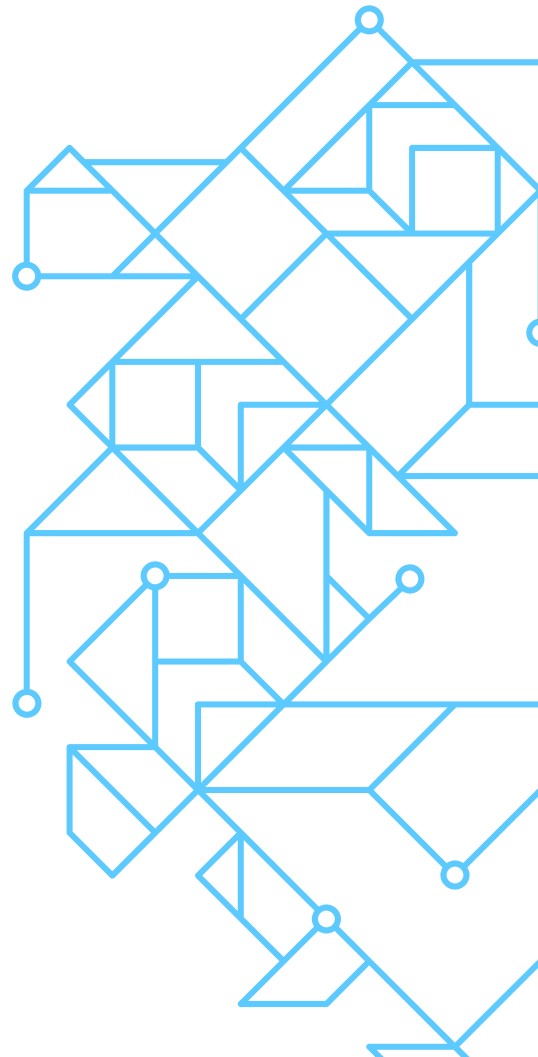
Only X's id (`id_X`) is
returned, not X's value

Distributed object store



`g(id_X)` is scheduled on same node, so `X` is never transferred

Examples of Distributed Applications with Ray



Distributed Applications with Ray

ML Libraries

- Ray AI Runtime
- Distributed scikit-learn/Joblib
- Distributed XGBoost on Ray
- Ray Multiprocess Pool

All using Ray core APIs & patterns

Experimenting & Monitoring Services

- WhyLabs
- Arize AI
- W & B
- MLflow

All using Ray core APIs & patterns

ML Platforms & Integrations

- Merlin (Shopify)
- Meta (TorchX)
- AirFlow, Prefect
- HuggingFace
- Ludwig AI
- Uber
- Instacart
- Spotify

All using Ray core APIs & patterns